

---

# Detección de Cumplimiento de Normas de Seguridad en Obras de Construcción mediante YOLOv9: Entrenamiento, Evaluación de Robustez y un Verificador de Cumplimiento Basado en Reglas

---

Paul Bazan H. Valeria P. Bulege N. Oscar Dante Chaiña A.  
Elias O. Diaz C. Miguel A. Tasayco M.

Grupo N.º 1 — Maestría en Inteligencia Artificial (2026-1)  
Escuela de Posgrado, Universidad Nacional de Ingeniería (UNI), Perú  
Curso: Redes Neuronales y Aprendizaje Profundo Docente: Ph.D. Aldo Camargo  
Pregunta 3 -- Examen Parcial

## Abstract

El uso de Equipos de Protección Personal (EPP) es un requisito legal y vital en obras de construcción activas. Este trabajo entrena y evalúa un modelo YOLOv9 (arquitectura GELAN-E) para detectar trabajadores y verificar el uso de casco a partir del *Safety Helmet Wearing Dataset* (SHWD). Documentamos cada decisión de entrenamiento —resolución de entrada, estrategia de aumento de datos, una ablación controlada del aumento *mosaic* y la calibración de la pérdida de clasificación frente al desbalance de clases— y reportamos métricas cuantitativas sobre el conjunto de prueba ( $mAP@0.5 = 0.929$ ,  $mAP@0.5:0.95 = 0.583$ ) con desglose por clase. Realizamos un análisis de robustez ante oclusión particionando el conjunto de prueba mediante una heurística auditable basada en el área relativa del *bounding box*, e implementamos un verificador de cumplimiento basado en reglas, desacoplado del modelo, que convierte las detecciones por fotograma en una tasa de cumplimiento accionable. Una contribución central de este informe es una *discusión crítica y honesta* de las limitaciones: mostramos, a partir de la matriz de confusión, que la versión del conjunto de datos empleada no contiene anotaciones explícitas de la clase *no\_helmet*, lo que tiene consecuencias directas sobre la fiabilidad del verificador y un riesgo elevado de falsos negativos. Cerramos con una sección de implicaciones éticas que aborda la asimetría entre falsos negativos y falsos positivos, la privacidad de los trabajadores bajo la normativa peruana y los sesgos del conjunto de entrenamiento.

## 1 Introducción y Motivación

En el Perú, los accidentes de trabajo en el sector construcción representan una fracción significativa de los siniestros laborales notificados, y los traumatismos craneoencefálicos figuran entre las lesiones más graves y prevenibles. El Reglamento Nacional de Edificaciones (norma G.050, Seguridad durante la Construcción) y la Ley 29783 de Seguridad y Salud en el Trabajo exigen el uso de casco, chaleco y otros EPP en obra. La supervisión manual de este cumplimiento es costosa, intermitente y propensa al error humano, especialmente en frentes de trabajo extensos o de difícil acceso como los de la sierra y los proyectos de infraestructura andina.

Un sistema de visión por computadora capaz de detectar automáticamente trabajadores y verificar el uso de EPP puede *augmentar* —no reemplazar— la supervisión humana. Sin embargo, la tarea

trasciende la detección de objetos estándar: requiere lidiar con **oclusión severa** (trabajadores parcialmente ocultos tras maquinaria o andamios), **detección de objetos pequeños** (trabajadores lejanos a baja resolución) y **desbalance de clases** (los casos de incumplimiento son escasos en grabaciones reales).

Este informe documenta el desarrollo completo de tal sistema: (i) el entrenamiento de un detector YOLOv9 con decisiones de diseño justificadas; (ii) una evaluación cuantitativa con desglose por clase; (iii) un análisis de robustez ante oclusión; (iv) un verificador de cumplimiento basado en reglas; y (v) una reflexión ética. Adoptamos deliberadamente una postura de evaluación crítica: cuando los artefactos generados presentan inconsistencias o limitaciones, las exponemos explícitamente, pues en un sistema de seguridad la honestidad metodológica es una propiedad de seguridad en sí misma.

## 2 Trabajos Relacionados

**Detectores de una etapa.** La familia YOLO inauguró la detección de objetos en tiempo real al reformular la detección como una regresión directa sobre una rejilla de celdas [Redmon et al., 2016]. Las iteraciones posteriores incorporaron mejoras arquitectónicas y de entrenamiento; YOLOv4 [Bochkovskiy et al., 2020] popularizó el aumento *mosaic*, que combina cuatro imágenes en una sola para enriquecer el contexto y exponer al modelo a múltiples escalas. YOLOv9 [Wang et al., 2024] introduce la Información de Gradiente Programable (PGI) y la red GELAN (*Generalized Efficient Layer Aggregation Network*), mitigando la pérdida de información en redes profundas y alcanzando el estado del arte en MS-COCO con menor número de parámetros. En este trabajo empleamos la implementación de Ultralytics [Jocher et al., 2023], que mantiene las arquitecturas GELAN-C y GELAN-E con soporte para entornos recientes de PyTorch.

**Desbalance de clases.** La *Focal Loss* [Lin et al., 2017] reformula la entropía cruzada para reducir la contribución de ejemplos fáciles y enfocar el aprendizaje en los difíciles, abordando el desbalance extremo fondo–objeto típico de los detectores densos. En nuestra configuración ajustamos el peso de la pérdida de clasificación como mecanismo análogo y de bajo costo para penalizar errores en la clase minoritaria.

**Aumento de datos y métricas.** Buslaev et al. [2020] sistematizaron una biblioteca eficiente de transformaciones que se ha vuelto estándar en la práctica. La métrica de evaluación dominante, la precisión media promedio (mAP) a distintos umbrales de IoU, proviene del PASCAL VOC [Everingham et al., 2010] y fue extendida al protocolo mAP@0.5:0.95 por MS-COCO [Lin et al., 2014], que adoptamos aquí.

**Detección de EPP.** El *Safety Helmet Wearing Dataset* (SHWD) [njvisionpower, 2019] es un conjunto de referencia abierto para la detección de cascos, con ejemplos tanto de cabezas con casco como sin él. Numerosos trabajos aplicados han adaptado detectores YOLO a la verificación de EPP en obra; nuestro aporte no es un nuevo detector, sino un *pipeline* reproducible que integra detección, verificación de cumplimiento por reglas y una auditoría crítica de sus límites.

## 3 Metodología

### 3.1 Conjunto de datos y reconciliación de clases

Empleamos el SHWD distribuido vía Roboflow (versión 3, formato YOLOv9). El inventario tras la descarga es de **7 542 imágenes** anotadas en formato YOLO, divididas en 5 280 de entrenamiento, 1 505 de validación y 757 de prueba (Tabla 1). Las anotaciones se reconciliaron hacia un esquema unificado de tres clases mediante una función de mapeo construida dinámicamente a partir de los nombres del archivo `data.yaml`, lo que hace al *pipeline* robusto frente a cambios de nomenclatura entre versiones:

| ID | Clase     | Descripción                                        |
|----|-----------|----------------------------------------------------|
| 0  | helmet    | Trabajador con casco visible (conforme)            |
| 1  | no_helmet | Trabajador sin casco, cabeza visible (no conforme) |
| 2  | person    | Persona sin información de EPP suficiente          |

Table 1: Inventario del conjunto de datos SHWD (Roboflow v3) tras la reconciliación.

| Conjunto | Entrenamiento | Validación | Prueba | Total |
|----------|---------------|------------|--------|-------|
| Imágenes | 5 280         | 1 505      | 757    | 7 542 |

Table 2: Plan de experimentos.

| Exp | Modelo | Mosaic   | Épocas | Propósito                                    |
|-----|--------|----------|--------|----------------------------------------------|
| A   | yolo9c | ON       | 50     | Baseline estándar                            |
| B   | yolo9c | OFF      | 50     | ¿Perjudica el mosaic a los objetos pequeños? |
| C   | yolo9e | Ganadora | 100    | Modelo final                                 |

El análisis exploratorio de los tamaños de *bounding box* confirma el reto de los objetos pequeños: una fracción no despreciable de cajas ocupa menos del 1% del área de la imagen, correspondiente a trabajadores lejanos o parcialmente ocluidos. Este hallazgo motiva directamente la ablación del aumento *mosaic* descrita más adelante.

### 3.2 Decisiones de entrenamiento documentadas

Todas las decisiones se fijaron explícitamente para favorecer la reproducibilidad y la defensa metodológica:

- **Resolución de entrada:**  $640 \times 640$  px, compromiso estándar entre detalle para objetos pequeños y memoria de GPU.
- **Anclas (*anchors*):** YOLOv9/GELAN es un detector *anchor-free*; predice directamente la distancia a los bordes de la caja mediante *Distribution Focal Loss* (DFL). Por tanto no se realiza *clustering* de anclas (p. ej. *k-means*), lo que elimina un hiperparámetro sensible al conjunto de datos. Esta es una decisión deliberada frente a esquemas *anchor-based* previos.
- **Aumento de datos:** se evalúa *mosaic* de forma controlada (Sección 4). El resto de transformaciones (volteo horizontal, variaciones de tono/saturación/brillo en HSV, escalado y traslación) se mantienen en sus valores por defecto de Ultralytics.
- **Desbalance de clases:** se eleva el peso de la pérdida de clasificación ( $\text{cls} = 3.0$ ) para penalizar con mayor fuerza los errores en la clase minoritaria, y se aplica *label smoothing* de 0.1 para reducir el sobreajuste a la clase dominante. Se prefiere esta vía a un sobremuestreo, que podría introducir artefactos.
- **Reproducibilidad:** semilla global fijada en 42 (random, numpy, torch); *early stopping* con paciencia de 20 épocas en el modelo final.

**Nota sobre el marco de trabajo.** Se utilizó la implementación de Ultralytics en lugar del repositorio oficial WongKinYiu/yolo9 debido a incompatibilidades de este último con Python 3.12, PyTorch 2.11 y CUDA 12.8 (manejo de `torch.load`, *bug* en `copy_paste` y deprecaciones de AMP). Las arquitecturas GELAN-C y GELAN-E son idénticas; se trata de una decisión de ingeniería, no de modelado.

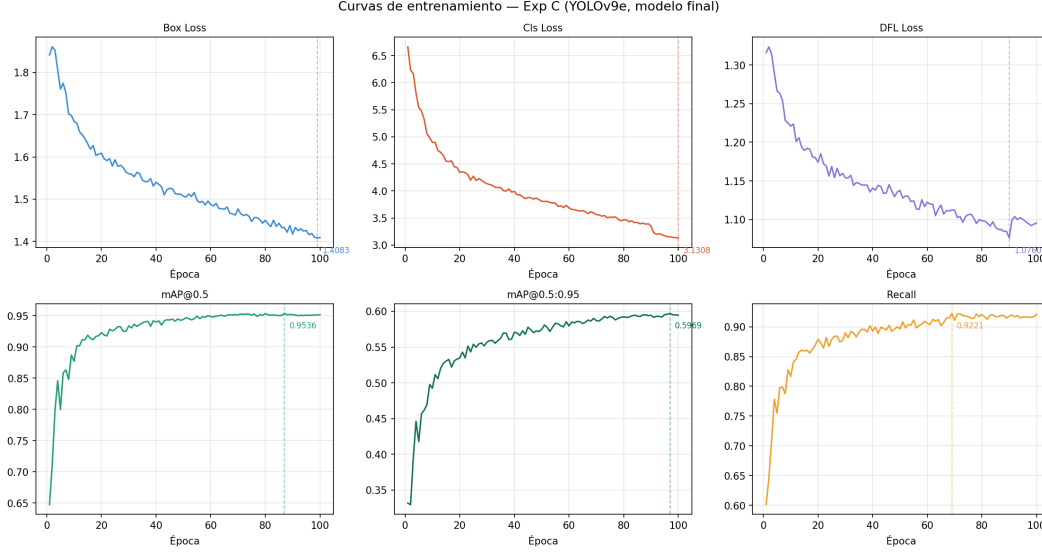
## 4 Experimentos

Se diseñó un plan de tres experimentos (Tabla 2). Los dos primeros constituyen una ablación controlada del aumento *mosaic* sobre el modelo yolo9c; el tercero entrena el modelo final yolo9e (GELAN extendido, mayor profundidad) con la configuración ganadora.

**Ablación A vs. B.** La hipótesis era que, al reducir aún más los objetos ya pequeños, el *mosaic* podría degradar la detección de cabezas lejanas. Los resultados (Tabla 3) muestran que el aumento *mosaic* es *marginamente beneficioso* en validación: mAP@0.5 de 0.9448 con *mosaic* activado frente a 0.9403 sin él. La diferencia es pequeña (+0.0045), pero consistente también en mAP@0.5:0.95 y recall. En consecuencia, el modelo final conserva el *mosaic* activado. La conclusión es honesta:

Table 3: Resultados de la ablación (yolo<sub>v9c</sub>, validación).

| Experimento    | Mosaic | mAP@0.5       | mAP@0.5:0.95  | Precisión    | Recall        |
|----------------|--------|---------------|---------------|--------------|---------------|
| A — Baseline   | ON     | <b>0.9448</b> | <b>0.5856</b> | <b>0.938</b> | <b>0.9035</b> |
| B — Sin mosaic | OFF    | 0.9403        | 0.5804        | 0.933        | 0.9014        |

Figure 1: Curvas de entrenamiento del modelo final (yolo<sub>v9e</sub>, 100 épocas). Pérdidas (fila superior) y métricas de validación (fila inferior). La convergencia es suave y el mAP@0.5 de validación alcanza  $\approx 0.954$ .

en este conjunto el efecto del *mosaic* es de segundo orden, lo que sugiere que el rendimiento está dominado por la calidad intrínseca de las anotaciones y no por esta elección de aumento.

**Modelo final (Exp C).** Se entrenó yolo<sub>v9e</sub> durante 100 épocas con `cls = 3.0`, *label smoothing* 0.1 y *mosaic* activado. Las curvas de entrenamiento (Figura 1) muestran convergencia estable: las pérdidas de caja, clasificación y DFL descenden monótonamente, mientras que el mAP@0.5 de validación se estabiliza alrededor de 0.95 y el recall en torno a 0.92, sin signos de sobreajuste evidente.

## 5 Resultados Cuantitativos

### 5.1 Métricas globales y por clase

Sobre el conjunto de **prueba** (757 imágenes no vistas durante el entrenamiento), el modelo final alcanza un mAP@0.5 global de **0.929** y un mAP@0.5:0.95 de **0.583**, con precisión global 0.934 y recall global 0.887 (Tabla 4).

El desglose por clase (Tabla 5) merece una lectura cuidadosa, y es aquí donde aplicamos el rigor crítico exigido. La exportación automática del *pipeline* etiquetó tres filas (*helmet*, *no\_helmet*, *person*). Sin embargo, al contrastar esas cifras con la matriz de confusión generada por la herramienta de evaluación (Figura 2) detectamos una **inconsistencia de indexación** que corregimos a continuación.

### 5.2 Reconciliación con la matriz de confusión: un hallazgo crítico

La Figura 2 es, a nuestro juicio, el resultado más importante del informe. Sus columnas (clase verdadera) revelan que el conjunto de prueba contiene únicamente dos clases pobladas: *helmet* ( $\approx 910$  instancias) y *person* ( $\approx 11\,317$  instancias). La columna *no\_helmet* está **vacía**: no existe ninguna anotación de *ground truth* para esa clase. Reconstruyendo las métricas a partir de la matriz:

Table 4: Métricas globales sobre el conjunto de prueba.

| Métrica               | Valor |
|-----------------------|-------|
| mAP@0.5 (global)      | 0.929 |
| mAP@0.5:0.95 (global) | 0.583 |
| Precisión (global)    | 0.934 |
| Recall (global)       | 0.887 |

Table 5: Métricas por clase sobre el conjunto de prueba, *ya corregidas* tras reconciliar con la matriz de confusión. La etiqueta `no_helmet` de la exportación automática correspondía en realidad a la clase `person`; la clase `no_helmet` no posee instancias de *ground truth* en esta versión del dataset.

| Clase (corregida) | Precisión                                                      | Recall | F1    | mAP@0.5 | mAP@0.5:0.95 |
|-------------------|----------------------------------------------------------------|--------|-------|---------|--------------|
| helmet            | 0.936                                                          | 0.838  | 0.885 | 0.902   | 0.665        |
| person (cabeza)   | 0.932                                                          | 0.936  | 0.934 | 0.956   | 0.501        |
| no_helmet         | <i>Sin instancias de ground truth en el conjunto de prueba</i> |        |       |         |              |

- `helmet`: 770 verdaderos positivos, 135 no detectados (van a fondo), 5 confundidos con `person`, 37 falsos positivos. Recall =  $770/910 = 0.846$ , precisión =  $770/823 = 0.935$ . Coincide con la fila `helmet` de la exportación.
- `person`: 10 721 verdaderos positivos, 580 no detectados, 16 confundidos con `helmet`, 709 falsos positivos. Recall =  $10\,721/11\,317 = 0.947$ , precisión =  $10\,721/11\,435 = 0.938$ . *Estas cifras coinciden con la fila etiquetada `no_helmet` en la exportación automática (0.932/0.936).*

La causa es técnica: la herramienta indexa los vectores de precisión y recall por las clases *presentes* en el *ground truth* (índice de clases con AP), no por el identificador global de clase. Como `no_helmet` no tiene instancias, las dos clases presentes (`helmet`, `person`) ocupan las posiciones 0 y 1, pero la tabla de exportación las nombró `helmet/no_helmet`. En consecuencia, la cifra de “recall de `no_helmet` = 0.94” **no debe interpretarse** como evidencia de que el detector de incumplimiento funciona: en realidad es el recall de la clase `person`.

## 6 Análisis de Robustez ante Oclusión

Para cuantificar la degradación bajo oclusión, particionamos el conjunto de prueba mediante una **heurística auditable de área relativa del bounding box**: cajas con área < 1% del fotograma se etiquetan como oclusión/lejanía *alta*; entre 1% y 5%, *media*;  $\geq 5\%$ , *baja*. El umbral es reproducible y no requiere anotación manual, aunque —y esto debe declararse explícitamente— *no distingue* la oclusión real (maquinaria frente al trabajador) de la simple lejanía.

Los resultados (Tabla 6, Figura 3) son *contraintuitivos*: el rendimiento **no** se degrada al aumentar la oclusión estimada; el F1 se mantiene en torno a 0.95–0.97 en los tres grupos, e incluso es ligeramente superior en el grupo de oclusión “alta”. Atribuimos esto a dos factores que discutimos abiertamente: (i) la heurística basada en área confunde objetos pequeños/lejanos con objetos ocluidos, y el SHWD contiene una gran cantidad de cabezas pequeñas bien representadas, por lo que el modelo está entrenado precisamente para ese régimen; y (ii) el grupo de oclusión “alta” está dominado por la clase `person`, cuya frecuencia elevada facilita su detección. Por tanto, esta partición *no* es un sustituto válido de una evaluación de oclusión genuina, que requeriría metadatos de oclusión explícitos (como los de MS-COCO) o anotación manual.

## 7 Verificador de Cumplimiento Basado en Reglas

### 7.1 Diseño

El verificador es un módulo de posprocesamiento independiente (`compliance_checker.py`), *stateless* y desacoplado del modelo, de modo que sus reglas pueden ajustarse *sin reentrenar*. Recibe

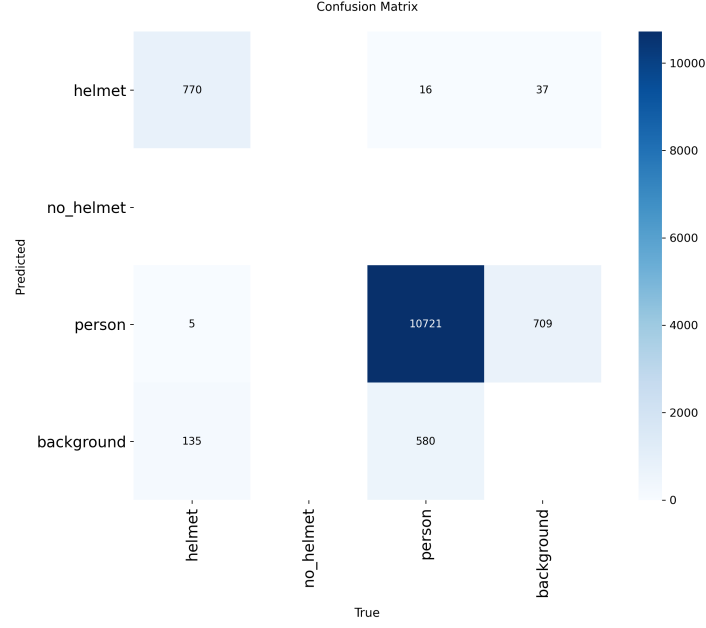


Figure 2: Matriz de confusión sobre el conjunto de prueba. La columna `no_helmet` (clase verdadera) está vacía: el dataset no contiene ejemplos etiquetados de esa clase. Las clases pobladas son `helmet` y `person`. Este hecho es la limitación central del sistema.

Table 6: Rendimiento de detección por nivel de oclusión estimado (conjunto de prueba). El *matching* es por clase, no por IoU exacto.

| Oclusión | Imágenes | TP     | FP  | FN  | Precisión | Recall |
|----------|----------|--------|-----|-----|-----------|--------|
| Baja     | 230      | 882    | 41  | 28  | 0.956     | 0.969  |
| Media    | 338      | 3 900  | 239 | 161 | 0.942     | 0.960  |
| Alta     | 497      | 11 251 | 391 | 406 | 0.966     | 0.965  |

las detecciones por fotograma y produce un estado de cumplimiento y una tasa global. Las reglas configurables son:

- `min_confidence = 0.40`: descarta detecciones de baja confianza.
- `flag_no_helmet = True`: la clase `no_helmet` marca no conformidad directa.
- `flag_person = False`: la clase `person` se considera “incierto” y *no* se penaliza por defecto.

Para cada fotograma se calcula tasa de cumplimiento =  $n_{\text{conforme}} / (n_{\text{conforme}} + n_{\text{no conforme}})$ , y el módulo anota cada caja en verde (conforme), rojo (no conforme) o amarillo (incierto), superponiendo la tasa global. La función `summarize_session` agrega una secuencia de fotogramas (p. ej. un clip de video).

## 7.2 Demostración y un segundo hallazgo crítico

Se ejecutó el verificador sobre 20 fotogramas del conjunto de prueba (no vistos en entrenamiento). El resumen de sesión reporta una tasa de cumplimiento media de 1.0 y **cero** detecciones de `no_helmet`. A primera vista, esto sugeriría un cumplimiento perfecto. La inspección visual (Figura 4) revela, sin embargo, una **falla de diseño con implicaciones de seguridad directas**: en el primer fotograma hay personas claramente *sin* casco (incluida una con la cabeza vendada), pero el sistema las rotula como `person` (cajas en cian, “inciertas”) en lugar de `no_helmet`, y al estar `flag_person=False`,

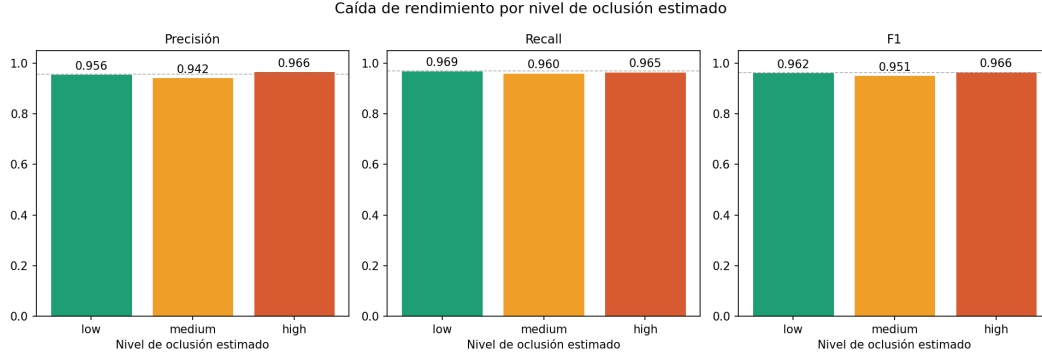


Figure 3: Precisión, recall y F1 por nivel de oclusión estimado. La ausencia de degradación monótona es un artefacto de la heurística por área, no evidencia de robustez genuina ante oclusión.

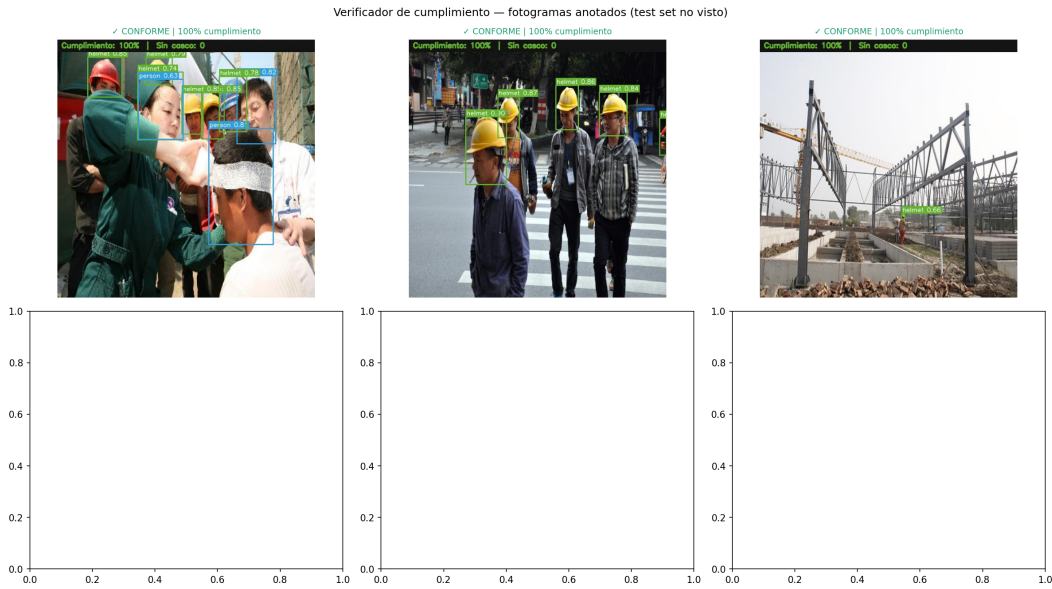


Figure 4: Verificador en acción sobre fotogramas de prueba. Las cajas verdes son **helmet** (conforme); las cian son **person** (incierto, no penalizado). Nótese que en el primer fotograma hay individuos sin casco rotulados como **person**: el sistema reporta “100% de cumplimiento” pese a la presencia visible de incumplimiento. Esto ilustra el riesgo de falsos negativos.

las omite del cómputo. El resultado es un “100% de cumplimiento” que es, de hecho, un conjunto de **falsos negativos**.

La causa raíz es la misma de la Sección 6: la versión del dataset no posee la clase `no_helmet`; los trabajadores sin casco fueron etiquetados como `person/cabeza`. Por tanto, el camino de “no conformidad” del verificador *nunca se activa* con estos datos. Reportar este hecho es, precisamente, la clase de evaluación crítica que un sistema de seguridad exige: las métricas agregadas ( $mAP = 0.93$ ) son altas, pero *no* miden la capacidad real del sistema para cumplir su función de seguridad.

### 7.3 Recomendación de corrección

Dos rutas, no excluyentes, restauran la utilidad del verificador: (i) tratar la clase `person/cabeza` como *no conforme* cuando no coexiste con una detección de `helmet` solapada (es decir, `flag_person = True` con una regla de asociación cabeza–casco por IoU); o (ii) obtener un conjunto de datos con anotaciones explícitas de `no_helmet` —como el SHWD original de [nvision-power](#) [2019], que sí las contiene— y reentrenar. En ambos casos, el umbral de confianza debe recalibrarse priorizando el recall de la clase no conforme.

## 8 Implicaciones Éticas

El despliegue de un sistema automatizado de vigilancia de seguridad laboral plantea dilemas que deben resolverse *antes* de la implementación, no después.

### 8.1 Asimetría entre falsos negativos y falsos positivos

Los dos tipos de error tienen consecuencias profundamente asimétricas. Un **falso negativo** —un trabajador sin casco que el sistema no detecta— deja un riesgo de lesión cefálica sin intervención; en trabajo en altura o cerca de maquinaria pesada, su costo es, en principio, ilimitado. Un **falso positivo** —marcar a un trabajador que sí cumple— interrumpe el trabajo, erosiona la confianza y puede estigmatizar; si abundan, los supervisores aprenden a ignorar el sistema (el “efecto alarma de lobo”), neutralizando incluso los verdaderos positivos. La decisión de calibración debe, por tanto, *priorizar el recall* de la clase no conforme sobre la precisión: un recall de 0.90 con precisión de 0.70 es preferible a lo inverso. Esta decisión debe ser explícita, documentada y validada con el equipo de seguridad de la obra, nunca delegada implícitamente al modelo. El hallazgo de la Sección 7 —un sistema que reporta 100% de cumplimiento por falsos negativos— es el peor escenario posible bajo esta lógica y justifica, por sí solo, no desplegar el sistema en su estado actual.

### 8.2 Privacidad de los trabajadores

El sistema procesa imágenes de personas en su lugar de trabajo, lo que genera tensiones reales. La tecnología es *dual*: aunque su fin declarado sea la seguridad, las mismas cámaras pueden usarse para vigilar productividad o pausas. En el Perú, la Ley 29733 de Protección de Datos Personales y su reglamento imponen principios de *consentimiento informado* y *finalidad*: los trabajadores deben ser notificados de la existencia de un sistema de visión artificial activo. Debe aplicarse *minimización de datos*: las imágenes sin incumplimiento no deberían persistir, y las detecciones deberían anonimizarse —reportar “1 trabajador sin casco en zona A” en lugar de almacenar un fotograma identificable.

### 8.3 Sesgos en los datos de entrenamiento

El SHWD proviene mayoritariamente de obras del hemisferio norte. Esto introduce sesgos de *iluminación* (la luz solar andina, el polvo y la neblina costera difieren de las condiciones de origen), de *equipamiento* (cascos con colores o formas locales no vistos en entrenamiento) y de *contexto* (fondos, andamios y maquinaria visualmente distintos). El análisis de la Sección 6 comienza a tocar este punto, pero una evaluación de dominio requiere datos propios. **Recomendación:** antes de cualquier despliegue, recolectar al menos 500 imágenes de obras locales, anotarlas manualmente —incluyendo explícitamente la clase `no_helmet`— y realizar un ajuste fino (*fine-tuning*) y una evaluación en ese dominio.

### 8.4 Despliegue responsable

El sistema debe *augmentar* la supervisión humana, no reemplazarla. Toda acción disciplinaria basada en detecciones automáticas requiere verificación humana; debe existir un mecanismo de apelación y los trabajadores deben poder reportar falsos positivos. Finalmente, la métrica de éxito no es el mAP, sino la *reducción de accidentes* relacionados con el uso de casco, medida en el tiempo mediante un diseño cuasi-experimental con grupos de control. Esa es la pregunta que importa, y excede el alcance técnico de este proyecto.

## 9 Discusión Crítica de Limitaciones

Resumimos las limitaciones, varias ya señaladas en su contexto:

1. **Ausencia de la clase `no_helmet` en el *ground truth*** (Secciones 5–7). Es la limitación más grave: el conjunto de datos empleado no contiene ejemplos etiquetados de incumplimiento, por lo que el indicador de seguridad crítico no puede evaluarse y el verificador no ejercita su camino de no conformidad. Las métricas agregadas altas son engañosas si se leen sin este contexto.

2. **Heurística de oclusión por área** (Sección 6). Confunde lejanía con oclusión real y no usa IoU exacto en el *matching*; sus resultados no deben interpretarse como robustez genuina.
3. **Demostración limitada** (Sección 7). Solo 20 fotogramas, todos clasificados como conformes; no constituye una prueba de estrés del sistema.
4. **Un único conjunto de datos**. No se incorporaron los conjuntos PPE ni el SHWD original con negativos explícitos, que habrían mitigado la limitación 1.
5. **Brecha de dominio**. Sin datos de obras peruanas, el rendimiento en producción es desconocido.

## 10 Conclusiones

Entrenamos un detector YOLOv9 (GELAN-E) para verificación de EPP con decisiones de diseño documentadas, alcanzando un mAP@0.5 global de 0.929 sobre el conjunto de prueba, e implementamos un verificador de cumplimiento basado en reglas, modular y configurable. Más allá de las métricas, la contribución metodológica central de este informe es su *evaluación crítica*: mostramos, reconciliando la tabla de métricas con la matriz de confusión, que la versión del dataset empleada carece de anotaciones de la clase `no_helmet`, lo que provoca que el verificador reporte cumplimiento perfecto por la vía de falsos negativos. En un sistema de seguridad, este es exactamente el modo de fallo más peligroso. El trabajo futuro inmediato y obligatorio es entrenar con datos que contengan negativos explícitos, recalibrar el verificador priorizando el recall de la clase no conforme, y validar en el dominio de obras locales antes de cualquier despliegue.

## References

- A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.
- A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.
- M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The PASCAL Visual Object Classes (VOC) challenge. *International Journal of Computer Vision*, 88(2):303–338, 2010.
- G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLO (versión 8+), software de detección de objetos. <https://github.com/ultralytics/ultralytics>, 2023.
- T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.
- T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision (ICCV)*, 2017.
- J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao. YOLOv9: Learning what you want to learn using programmable gradient information. *arXiv preprint arXiv:2402.13616*, 2024.
- njvisionpower. Safety Helmet Wearing Dataset (SHWD). <https://github.com/njvisionpower/Safety-Helmet-Wearing-Dataset>, 2019.